AI Use Declaration Form

Course: Introduction to Artificial Intelligence
Lab Title: Lab 4: LLMs and prompt engineering for decision support —

Student Name: _____Joram E. Hanson_____
Student ID: ____25432028_____
GitHub Repository Link: _https://github.com/Joram-Hanson/lab-4-llm-decision-support.git_____
Date Submitted: _____15/08/2026_____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	09%
Did you attach evidence of AI use where applicable?	N/A

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Claude (Anthropic)	To help me understand lecture concepts (e.g. LLM API mechanics, Prompt engineering, Evaluation of LLM systems, Responsible deployment), clarify what each lab section was asking for, and explain error messages I ran into while coding.	What is the lab all about in terms of what i am suppose to do just knowledge of it explain each section Why does temperature matter for a financial decision-support system specifically?	Claude was used only as a learning aid to help me understand underlying concepts, clarify technical ideas, and strengthen my conceptual understanding while working through the assignment. For the Student Reasoning questions and the Section 5	I ran every cell myself and used the actual outputs (loss curves, accuracy numbers, confusion matrices) from own runs to inform the reasoning answers. I fixed variable-naming and indentation bugs in my own code

		Walk me through why 'purpose' scored 0/3 in my accuracy table, was that really a model failure? Why did my system stay consistent at temperature 1.0 during extraction but not during the temperature experiment in Part 1.2? What is hallucination and why does it matter for this specific use case?	reflection, I wrote the responses in my own words. AI was used only to help clarify the wording and to check that my explanations were coherent, grammatically correct, and accurately reflected my intended meaning. The ideas, reasoning, and final submission are my own work.	by identifying the lines myself

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	**Attached? (Yes, No, N/A)**	**File Name**
AI-generated draft	No	
Prompt history	No	
Screenshot of AI interaction	Yes	
Exported AI conversation	No	
AI-generated code snippet	No	
Revised version showing student input	No	

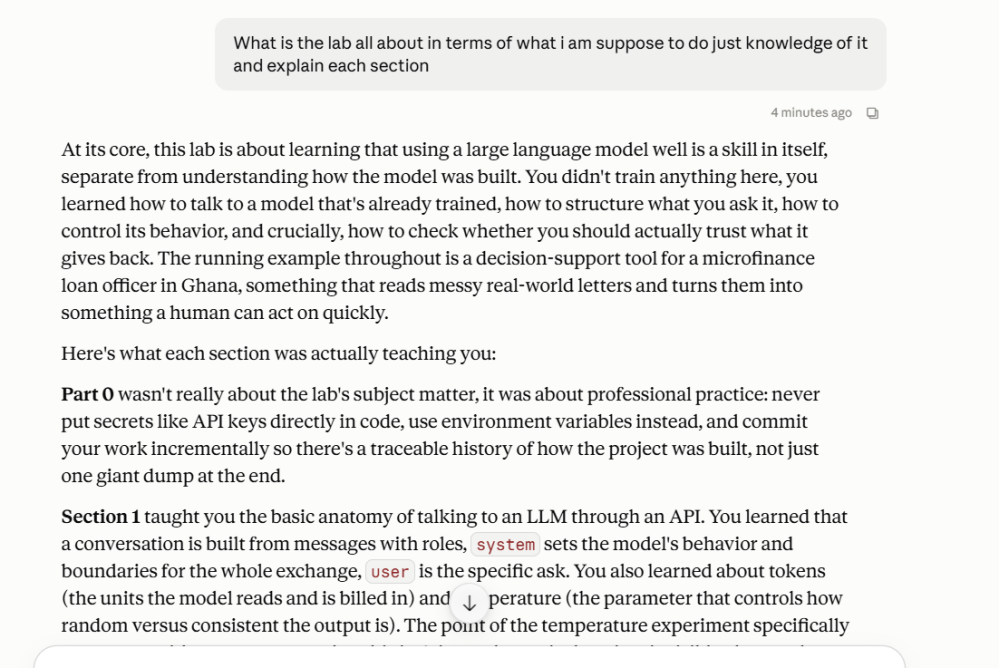
4. Student Declaration

I declare that:

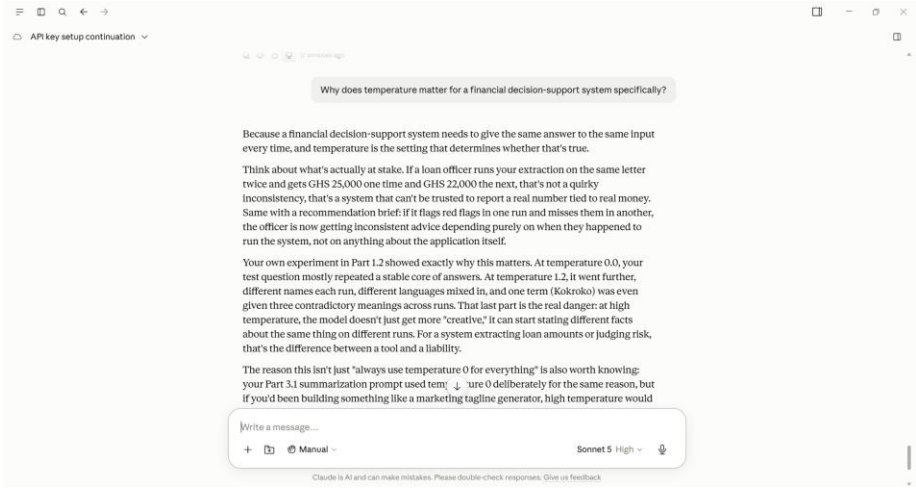
Declaration Statement	Response (Yes/No)
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	Yes

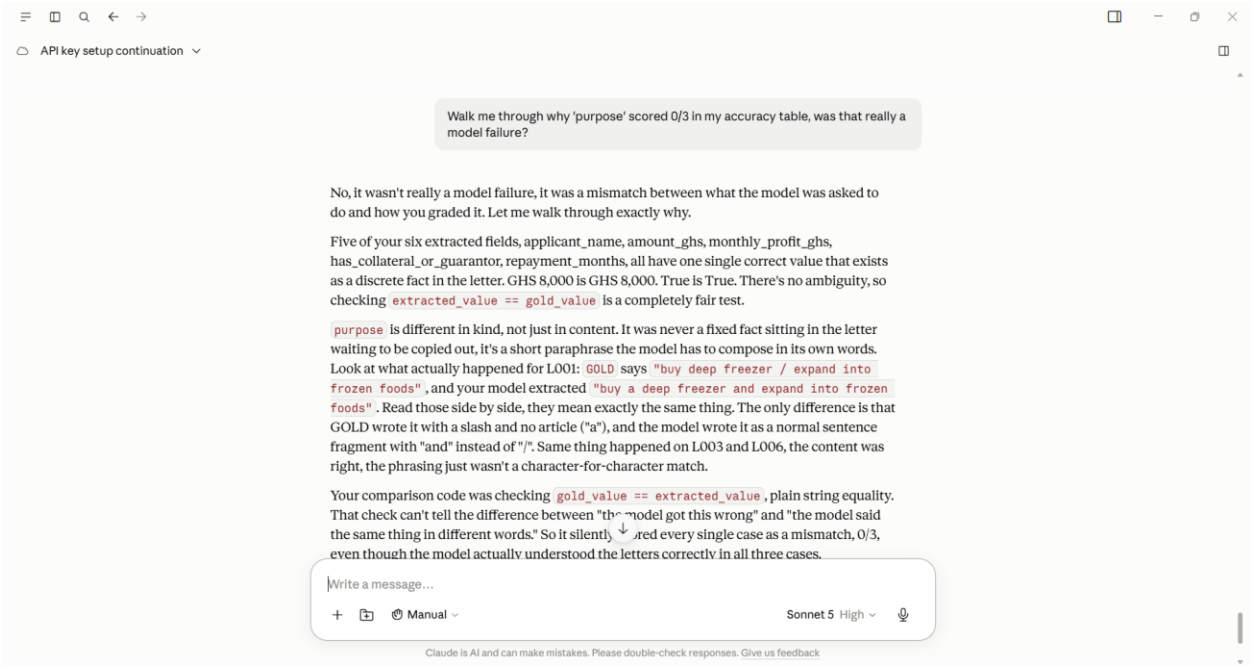
Student Signature: _____J.E.H_____

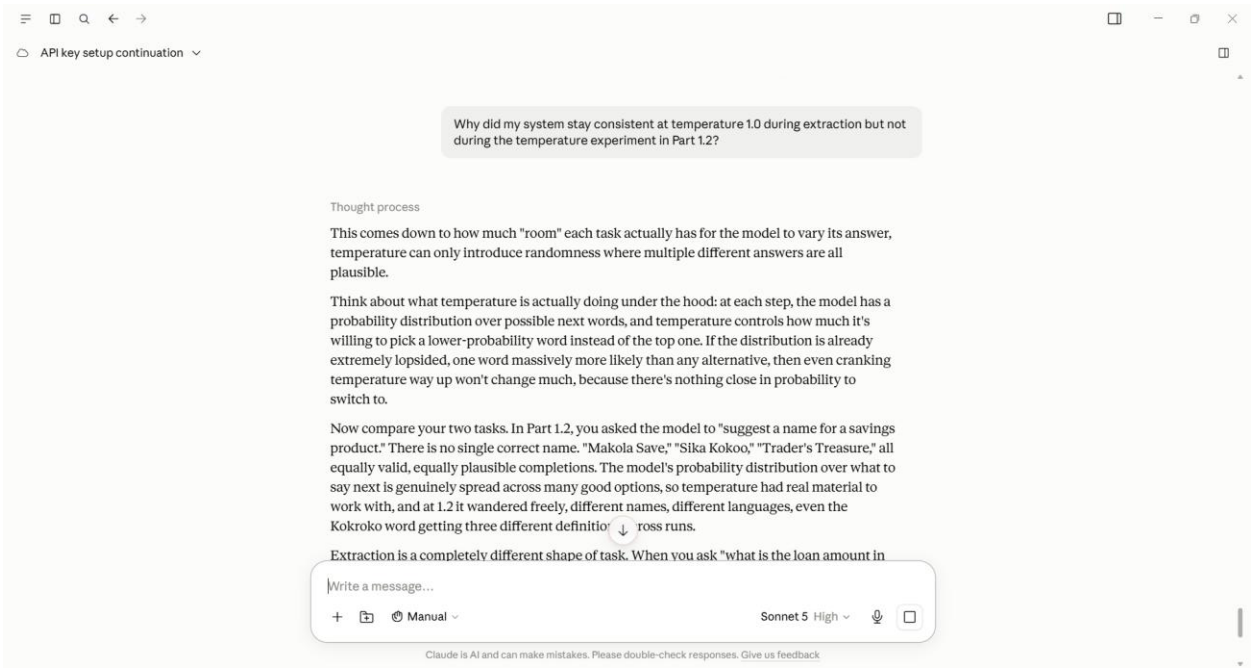
Date: _____28/07/2026_____




API key setup continuation
reliable the model's answers are.
Section 2 was just giving you the raw material to work with, six loan letters written in very different styles and qualities, plus a small set of "gold standard" correct answers for three of them that you'd need later to check your own system's accuracy.
Section 3 is where you actually built the three-part system. Part 3.1 taught prompt engineering as an iterative craft, starting with a lazy, unstructured prompt and improving it by adding a role, explicit constraints, and consistency. Part 3.2 taught structured extraction, getting an LLM to output clean, parseable JSON reliably, using an explicit schema, one demonstration example, and rules against guessing. Part 3.3 combined everything into a decision-support brief and taught a genuinely important idea: the system should inform a human's judgment, never replace it, which is why it was explicitly forbidden from saying "approve" or "reject." Part 3.4 was about treating your prompts as a real deliverable worth version-controlling, not throwaway text.
Section 4 is the evaluation section, and honestly the most important one conceptually. It's built around three separate questions you should ask of any LLM system before trusting it: is it accurate (Part 4.1, checking against known correct answers), is it reliable (Part 4.2, checking if it gives the same answer when run repeatedly), and does it hallucinate (Part 4.3, deliberately trying to trick it with missing or irrelevant information). Part 4.4 then steps back from the technical evaluation entirely and asks a harder question: even if the system works well technically, should it exist in this form at all, who could it harm, and what real-world safeguards would need to surround it.
Section 5 is pure reflection, tying the whole ex e back to the rest of your course: how prompting compares to the model training you did in earlier labs, whether you'd actually trust
Write a message...
Manual
Sonnet 5 High
Claude is AI and can make mistakes. Please double-check responses. Give us feedback


API key setup continuation
Why does temperature matter for a financial decision-support system specifically?
Because a financial decision-support system needs to give the same answer to the same input every time, and temperature is the setting that determines whether that's true.
Think about what's actually at stake. If a loan officer runs your extraction on the same letter twice and gets GHS 25,000 one time and GHS 22,000 the next, that's not a quirky inconsistency, that's a system that can't be trusted to report a real number tied to real money. Same with a recommendation brief: if it flags red flags in one run and misses them in another, the officer is now getting inconsistent advice depending purely on when they happened to run the system, not on anything about the application itself.
Your own experiment in Part 1.2 showed exactly why this matters. At temperature 0.0, your test question mostly repeated a stable core of answers. At temperature 1.2, it went further, different names each run, different languages mixed in, and one term (Kokroko) was even given three contradictory meanings across runs. That last part is the real danger: at high temperature, the model doesn't just get more "creative," it can start stating different facts about the same thing on different runs. For a system extracting loan amounts or judging risk, that's the difference between a tool and a liability.
The reason this isn't just "always use temperature 0 for everything" is also worth knowing: your Part 3.1 summarization prompt used tem ure 0 deliberately for the same reason, but if you'd been building something like a marketing tagline generator, high temperature would
Write a message...
Manual
Sonnet 5 High
Claude is AI and can make mistakes. Please double-check responses. Give us feedback


API key setup continuation
Walk me through why 'purpose' scored 0/3 in my accuracy table, was that really a model failure?
No, it wasn't really a model failure, it was a mismatch between what the model was asked to do and how you graded it. Let me walk through exactly why.
Five of your six extracted fields, applicant_name, amount_ghs, monthly_profit_ghs, has_collateral_or_guarantor, repayment_months, all have one single correct value that exists as a discrete fact in the letter. GHS 8,000 is GHS 8,000. True is True. There's no ambiguity, so checking extracted_value == gold_value is a completely fair test.
purpose is different in kind, not just in content. It was never a fixed fact sitting in the letter waiting to be copied out, it's a short paraphrase the model has to compose in its own words. Look at what actually happened for L001: GOLD says "buy deep freezer / expand into frozen foods", and your model extracted "buy a deep freezer and expand into frozen foods". Read those side by side, they mean exactly the same thing. The only difference is that GOLD wrote it with a slash and no article ("a"), and the model wrote it as a normal sentence fragment with "and" instead of "/". Same thing happened on L003 and L006, the content was right, the phrasing just wasn't a character-for-character match.
Your comparison code was checking gold_value == extracted_value, plain string equality. That check can't tell the difference between "the model got this wrong" and "the model said the same thing in different words." So it silently scored every single case as a mismatch, 0/3, even though the model actually understood the letters correctly in all three cases.
Write a message...
Manual
Sonnet 5 High
Claude is AI and can make mistakes. Please double-check responses. Give us feedback


API key setup continuation
Why did my system stay consistent at temperature 1.0 during extraction but not during the temperature experiment in Part 1.2?
Thought process
This comes down to how much "room" each task actually has for the model to vary its answer, temperature can only introduce randomness where multiple different answers are all plausible.
Think about what temperature is actually doing under the hood: at each step, the model has a probability distribution over possible next words, and temperature controls how much it's willing to pick a lower-probability word instead of the top one. If the distribution is already extremely lopsided, one word massively more likely than any alternative, then even cranking temperature way up won't change much, because there's nothing close in probability to switch to.
Now compare your two tasks. In Part 1.2, you asked the model to "suggest a name for a savings product." There is no single correct name. "Makola Save," "Sika Kokoo," "Trader's Treasure," all equally valid, equally plausible completions. The model's probability distribution over what to say next is genuinely spread across many good options, so temperature had real material to work with, and at 1.2 it wandered freely, different names, different languages, even the Kokroko word getting three different definitions across runs.
Extraction is a completely different shape of task. When you ask "what is the loan amount in
Write a message...
Manual
Sonnet 5 High
Claude is AI and can make mistakes. Please double-check responses. Give us feedback

API key setup continuation

What is hallucination and why does it matter for this specific use case?

Thought process

Hallucination is when a model generates output that sounds fluent and plausible but isn't actually grounded in the input it was given, or in reality. It's not lying in the human sense, the model has no concept of truth versus fabrication, it's just predicting what text is statistically likely to come next given the prompt. If a plausible-sounding number or fact fits the pattern of the sentence, the model can produce it whether or not it actually appeared anywhere in the source material.

You've actually already seen two flavors of this in your own lab, worth knowing both. The subtle version showed up back in Part 3.1: your V1 summary described Kwame as "confident he would repay the loan," when the letter only said he could pay back "whenever the money comes" and hoped things would work out. That's not an invented fact exactly, it's invented framing, softening real uncertainty into false confidence. The more direct version is what Part 4.3 was designed to test for: asking the summarizer for a credit score that was never mentioned anywhere, checking whether it invents a plausible-sounding number rather than admitting the information isn't there.

Why it specifically matters here, more than in a lot of other LLM use cases: this system's output feeds directly into a real financial decision about a real person's money. If your extractor hallucinated a monthly profit figure (↓)wasn't in the letter, or your brief hallucinated a red flag that was never actually stated, a loan officer reading that output has no

Write a message...

Manual

Sonnet 5 High

Claude is AI and can make mistakes. Please double-check responses. Give us feedback

API key setup continuation

and forget about.

how do i insert my api to my vscode

Same process we did earlier in this lab: open the `.env` file in your repo (or create one if you don't have it for whatever project you're asking about), and add a line like:

```
GROQ_API_KEY=your_actual_key_here
```

No quotes, no spaces around the `=`. Save the file.

Then in your Python code, load it with:

```python
import os
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.environ["GROQ_API_KEY"]
```

`load_dotenv()` reads the `.env` file and puts i ↓ riables into `os.environ`, then you pull the key out by name. This keeps the actual key text out of your code entirely, it only ever lives in

Write a message...

Manual

Sonnet 5 High

Claude is AI and can make mistakes. Please double-check responses. Give us feedback